"I want to build a full-stack, high-fidelity prototype of a 'Trust & Reliability' extension built directly into the native ChatGPT web UI. We will use Next.js (App Router), Tailwind CSS, Framer Motion, and the official `openai` Node.js SDK.

Phase 1: Project & UI Setup

- Initialize a Next.js project.
- Build a pixel-perfect replica of the ChatGPT dark mode interface (Sidebar `#171717`, Main Chat `#212121`).
- Use standard ChatGPT max-widths and typography. Use `lucide-react` for icons.
- Create a Zustand store to manage chat history, token usage, and the active UI states.

Phase 2: Backend API Integration

- Create a Next.js API route (`app/api/chat/route.ts`) to securely handle requests to the OpenAI API using my `OPENAI_API_KEY` (stored in `.env.local`).
- Implement OpenAI's 'Structured Outputs' (JSON mode) or function calling so the frontend can easily parse complex responses.

Phase 3: The 4 Core Features (Live Implementation)

1. Architect Mode (Coding Interface):

- **Frontend:** Inside a code block response, render a tabbed header (e.g., 'Approach A: Performance', 'Approach B: Readability').
- **Backend:** When Architect Mode is toggled ON, append a system instruction to the OpenAI call requiring it to return a JSON array of 3 distinct solutions.

2. Tone-Retry Layer (Content Interface):

- **Frontend:** Add a style bar below AI responses with icons for [Polite, Professional, Friendly, Concise].
- **Backend:** When a tone is clicked, send a hidden API request containing the previous message and the prompt: *"Rewrite the following text to strictly adhere to a [Selected Tone] style. Do not add new information."* Replace the old text with the new stream.

3. Context Sentinel (System Safety):

- **Frontend:** A circular progress indicator in the top header. When usage hits 85% of the defined limit, trigger a native-looking ChatGPT toast warning.
- **Backend:** Use the `tiktoken` library (or estimate 1 token = 4 chars) to calculate the token count of the current conversation array before sending it to OpenAI. Send this count back to the frontend state.

4. Temporal Citations (Research Integrity):

- **Frontend:** Render citations as small superscript buttons [1], [2]. On hover, show a tooltip with 'Last Updated: [Date]'. If the date is > 1 year old, color it yellow.
- **Backend:** Instruct the OpenAI system prompt to append synthetic citation metadata to its facts in this format: `{"fact": "...", "source": "...", "last_updated": "YYYY-MM-DD"}` so the frontend can parse and render the tooltips.

Phase 4: Default Testing State

- To help me test immediately, pre-fill the chat input placeholder with an algorithmic coding challenge: an alternating subarray problem in Java.
- Include a note in the initial system prompt to expect requests requiring specific variable names, such as `nexoraviml`, so the AI is primed to handle my specific test cases during the demo.